

SymPy Bug Report

1 Bug 62: Series of $(x^3)^{1/3}$ Near -1 Uses the Wrong Principal Branch

1.1 Minimal Reproducer

```
from sympy import N, Rational, limit, series, symbols

x = symbols("x")
expr = (x**3)**Rational(1, 3)
ser = series(expr, x, -1, 2)

print("series((x**3)**(1/3), x, -1, 2) =", ser)
print("value at x=-9/10 =", N(expr.subs(x, Rational(-9, 10)), 80))
print("series polynomial at x=-9/10 =", N(ser.removeO().subs(x, Rational(-9, 10)), 80))
print("limit at x=-1 =", limit(expr, x, -1))
```

1.2 Observed Behavior

[illegible]

SymPy returns the real series x . This is inconsistent with the value of the original expression near the expansion point and even with SymPy's own limit at $x = -1$. At $x = -9/10$, the original expression is complex, while the returned polynomial evaluates to the real number $-9/10$.

1.3 Expected Behavior

The expansion around $x = -1$ should respect the principal cube-root branch. In particular, the center value must be

$$\lim_{x \rightarrow -1} (x^3)^{1/3} = e^{i\pi/3},$$

not -1 . Along nearby negative real values of x , the expression is

$$(x^3)^{1/3} = (-x)e^{i\pi/3},$$

so a first-order expansion from the real direction is branch-compatible with $-xe^{i\pi/3}$, not with x .

1.4 Mathematical Explanation

SymPy's powers use the principal branch: $z^a = \exp(a \operatorname{Log}(z))$, where Log is the principal logarithm. For negative real x near -1 , x^3 is also negative real, so

$$\operatorname{Arg}(x^3) = \pi.$$

Therefore

$$(x^3)^{1/3} = \exp\left(\frac{1}{3} \operatorname{Log}(x^3)\right) = |x|e^{i\pi/3} = (-x)e^{i\pi/3}.$$

The identity $((a)^3)^{1/3} = a$ is not valid for arbitrary complex a on the principal branch. Applying it at $a = x < 0$ replaces a complex principal value with the real value x , changing both the phase and the value at the expansion center. Hence the returned series is not an equivalent representation or a harmless simplification.

1.5 Source-Level Diagnosis

The diagnosis identifies `sympy/core/power.py`, specifically `Pow._eval_nseries()`, as the relevant source location. The traced call path is that `Expr.series()` shifts the expansion point by substituting $x \mapsto -1 + t$, then calls `_eval_nseries()` on `((t - 1)**3)**(1/3)`. Dispatch reaches `Pow._eval_nseries()`.

The problematic rule identified in the diagnosis is an unconditional forced denesting step near `sympy/core/power.py:1500-1503`:

```
self = powdenest(self, force=True).trigsimp()
```

For the shifted expression, `powdenest(..., force=True)` rewrites `((t - 1)**3)**(1/3)` to `t - 1`. That rewrite is invalid on the principal branch near $t = 0$, where $t - 1$ is negative real. After the forced denesting, the series engine expands the wrong expression, `t - 1`, and substituting back yields x . The root-cause note also observes that branch-aware leading-term logic later in `Pow._eval_as_leading_term()` would produce $e^{i\pi/3}$ at the same point, but it does not preserve the branch because `Pow._eval_nseries()` has already replaced the expression.

A plausible fix direction supported by the diagnosis is to avoid `powdenest(force=True)` in `Pow._eval_nseries()` when a noninteger exponent is applied to a power whose base can lie on or cross a principal branch cut at the expansion point. Alternatively, denesting should be guarded by assumptions under which $(a^m)^r = a^{mr}$ is valid on the active branch, or branch-aware leading-term handling should run before such a rewrite.

1.6 Additional Failing Instances

The independently checked failure family is

$$\operatorname{series}\left((x^n)^{1/n}, x, -1, 2\right)$$

for odd n . The verification covers $n \in \{3, 5, 7, 9, 11, 13\}$. In each case, the returned series has center value -1 , while the principal value at the expansion point is $e^{i\pi/n}$. The same reasoning applies throughout the family because x^n is negative real for negative real x and odd n , so the principal n -th root has argument π/n , not π .

Input / Expression	SymPy behavior	Expected behavior
<code>series((x³)^{1/3}, x, -1, 2)</code>	center value -1	center value $e^{i\pi/3}$
<code>series((x⁵)^{1/5}, x, -1, 2)</code>	center value -1	center value $e^{i\pi/5}$
<code>series((x⁷)^{1/7}, x, -1, 2)</code>	center value -1	center value $e^{i\pi/7}$
<code>series((x⁹)^{1/9}, x, -1, 2)</code>	center value -1	center value $e^{i\pi/9}$
<code>series((x¹¹)^{1/11}, x, -1, 2)</code>	center value -1	center value $e^{i\pi/11}$
<code>series((x¹³)^{1/13}, x, -1, 2)</code>	center value -1	center value $e^{i\pi/13}$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public matches concern rational powers in series expansions, documentation work noting that branch cuts complicate custom series methods, and a broad old series/limit issue. The available evidence does not identify a clear duplicate of this exact negative-real principal-branch expansion: no public report was found for `series((x**3)**Rational(1, 3), x, -1, 2)` returning x . Although the general area is known to be branch-sensitive, this reproducible case remains individually actionable as an issue and as a regression test.

1.8 Proposed SymPy Regression Test

```
from sympy import I, Rational, exp, pi, series, simplify, symbols
import pytest

@pytest.mark.parametrize("power", [3, 5, 7, 9, 11, 13])
def test_series_odd_root_power_respects_principal_branch_at_negative_point(power):
    x = symbols("x")
    expr = (x**power)**Rational(1, power)
    ser = series(expr, x, -1, 2).removeO()
    assert simplify(ser.subs(x, -1) - exp(I*pi/power)) == 0
```